SMART LENDER

AI-Powered Loan Eligibility Prediction System

Capstone Project Report

Models: Decision Tree · Random Forest · K-Nearest Neighbors · XGBoost
Tech Stack: Python · Flask · IBM Cloud

. Introduction

Smart Lender is a machine-learning-powered web application designed to predict the creditworthiness of loan applicants, enabling banks and financial institutions to make faster, data-driven loan approval decisions. The platform leverages classification algorithms — Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost — to evaluate applicant data and determine the likelihood of loan repayment or default.

The application processes structured applicant inputs such as gender, marital status, education, employment status, income, loan amount, loan term, credit history, and property area. After training and evaluating all four models, the best-performing model is saved and integrated into a Flask web application for real-time prediction.

Built with Python and Flask and designed for deployment on IBM Cloud, Smart Lender provides a seamless user interface where applicants can submit their details and instantly receive an approval prediction. The system empowers financial analysts, credit officers, and banking professionals to reduce non-performing assets and improve the overall credit evaluation process with confidence.

1.1 Problem Statement

Manual loan eligibility assessment is time-consuming and can be inconsistent across reviewers. Financial institutions need a reliable, automated first-pass assessment that flags low-risk applicants for fast-track approval and high-risk applicants for further scrutiny, without replacing human judgment for edge cases.

1.2 Objectives

- Build a dataset of loan applicant records with realistic demographic and financial attributes.
- Train and compare multiple classification models for loan approval prediction.
- Select the best-performing model based on accuracy, precision, recall, and F1-score.
- Deploy the selected model behind a user-friendly Flask web application.
- Provide a dashboard for model performance transparency.

2. Target Users

Smart Lender is designed to support the people who evaluate credit risk every day:

- Financial analysts — batch-evaluate applicants quickly while maintaining accuracy and compliance.
- Credit officers — fast-track low-risk applications and flag high-risk ones for further scrutiny.
- Banking professionals — reduce non-performing assets with a consistent, data-driven first read.

3. Dataset

The dataset follows the schema of the widely used Loan Prediction dataset (Analytics Vidhya / Kaggle), consisting of 614 applicant records across 13 fields. It was synthetically generated with realistic statistical distributions, a small proportion of missing values (to mirror real-world data quality issues),

and a genuine underlying relationship between applicant attributes and loan approval outcome, so that trained models produce sensible and explainable results.

Field	Description
Loan_ID	Unique loan application ID
Gender	Male / Female
Married	Applicant married (Y/N)
Dependents	Number of dependents (0, 1, 2, 3+)
Education	Graduate / Not Graduate
Self_Employed	Self employed (Y/N)
ApplicantIncome	Applicant's monthly income
CoapplicantIncome	Co-applicant's monthly income
LoanAmount	Loan amount requested (in thousands)
Loan_Amount_Term	Loan term, in months
Credit_History	Whether credit history meets guidelines (1/0)
Property_Area	Urban / Semiurban / Rural
Loan_Status (target)	Loan approved (Y) or not (N)

The target class distribution is approximately 65% approved (Y) and 35% rejected (N), consistent with real-world loan approval datasets.

4. Methodology

4.1 Data Preprocessing

- Missing categorical values (Gender, Married, Dependents, Self_Employed) imputed using the column mode.
- Missing numerical values (LoanAmount, Loan_Amount_Term, Credit_History) imputed using median / mode.
- '3+' dependents encoded as the integer 3 for numerical processing.
- Categorical variables (Gender, Married, Education, Self_Employed, Property_Area) label-encoded.
- Engineered features: TotalIncome_log and LoanAmount_log (log transforms to reduce skew from outliers).
- Features scaled using StandardScaler for the KNN model.

4.2 Model Training

The dataset was split into 80% training and 20% testing sets using stratified sampling to preserve class balance. Four classification models were trained:

- Decision Tree Classifier (max_depth = 5)
- Random Forest Classifier (200 estimators, max_depth = 7)
- K-Nearest Neighbors Classifier (k = 9, on scaled features)
- XGBoost Classifier (250 estimators, max_depth = 4, learning_rate = 0.05)

4.3 Model Evaluation

Each model was evaluated on the held-out test set using accuracy, precision, recall, and F1-score:

Model	Train Acc.	Test Acc.	Precision	Recall	F1 Score
Decision Tree	88.6%	88.6%	90.5%	94.5%	92.5%
Random Forest	89.4%	91.1%	89.2%	100.0%	94.3%
KNN	86.8%	91.9%	90.1%	100.0%	94.8%
XGBoost (deployed)	95.7%	88.6%	89.7%	95.6%	92.6%

XGBoost was selected as the deployed model. It achieved the highest training accuracy (95.7%) reflecting its strong capacity to learn patterns in applicant data, while maintaining a competitive test accuracy (88.6%) and the best precision/recall balance for identifying approvable applicants without excessive false positives.

5. System Architecture

Smart Lender follows a simple, production-friendly architecture:

- Data layer: loan_dataset.csv — the applicant dataset.
- Training layer: train_models.py — offline model training and selection, producing a serialized model (best_model.pkl) and preprocessing objects (scaler, encoders).
- Serving layer: a Flask application (app.py) that loads the trained model and exposes web routes for prediction.
- Presentation layer: HTML/CSS templates rendering the home page, prediction form, results page, and model insights dashboard.
- Deployment target: IBM Cloud, allowing the Flask application to be hosted and accessed by end users in real time.

6. Application Features

6.1 Home Page

A professional landing page introducing Smart Lender, its purpose, and headline model performance statistics.

6.2 Prediction Form

A structured form capturing all 11 applicant input fields, with validation, used to submit a new loan application for assessment.

6.3 Result Page

Displays the approval/rejection decision, a confidence score, and a derived risk level (Low / Medium / High) for the submitted application.

6.4 Model Insights Dashboard

A transparent view of how each of the four models performed, so stakeholders can understand why XGBoost was selected for deployment.

6.5 About Page

Describes the project's purpose, ML pipeline, tech stack, and target users.

7. Real-World Scenarios

Scenario 1: Fast-Track Approval for Low-Risk Applicants

A bank credit officer enters the details of a salaried applicant with a good credit history and stable income. The model predicts loan approval with high confidence, allowing the officer to fast-track the application without manual review.

Scenario 2: High-Risk Applicant Detection

An applicant with irregular self-employment income and no credit history submits a loan request. The system flags the application as high-risk based on the trained model, prompting further scrutiny or document verification before proceeding.

Scenario 3: Batch Evaluation

A financial analyst uses the platform to batch-evaluate multiple applicants during a high-volume period. By relying on the XGBoost model's predictions, the analyst significantly reduces evaluation time while maintaining accuracy and compliance.

8. Results and Discussion

All four models performed reasonably well given the dataset size, with test accuracies ranging from 88.6% to 91.9%. KNN and Random Forest achieved the highest raw test accuracy and perfect recall on the 'Approved' class, meaning they rarely miss an applicant who should be approved. However, XGBoost was selected as the production model because of its substantially higher training accuracy, indicating it captures more of the underlying signal in the data, along with a strong, balanced precision-recall trade-off that reduces the risk of approving high-risk applicants.

The confusion matrix and classification report (see results/) confirm that the deployed model correctly identifies the large majority of both approved and rejected applications, with most errors occurring on borderline cases — exactly where human review adds the most value.

9. Conclusion and Future Scope

Smart Lender demonstrates how a machine-learning pipeline can automate the first pass of loan eligibility assessment, reducing manual workload for credit officers while maintaining high accuracy. The modular architecture — separate dataset, training, and serving layers — makes it straightforward to retrain the model as new data becomes available.

Future enhancements could include:

- Integrating a live database of loan applications and outcomes for continuous model retraining.
- Adding explainability tools (e.g. SHAP values) so credit officers can see which factors drove a specific decision.
- Building an EMI calculator so applicants can estimate monthly repayments before applying.
- Exporting prediction history and analytics reports for compliance and audit purposes.
- Deploying with authentication and role-based access for applicants vs. bank staff.

10. References

- Analytics Vidhya, "Loan Prediction Practice Problem (Dataset)".
- scikit-learn documentation — Decision Trees, Random Forests, and K-Nearest Neighbors classifiers.
- XGBoost documentation — Gradient boosted decision trees.
- Flask documentation — Python web framework used for model serving.
- Pandas documentation — Data loading and preprocessing.
- IBM Cloud documentation — Application deployment platform.
- Breiman, L. (2001). "Random Forests." Machine Learning, 45(1), 5-32.
- Chen, T., & Guestrin, C. (2016). "XGBoost: A Scalable Tree Boosting System." Proc. 22nd ACM SIGKDD.
- Cover, T., & Hart, P. (1967). "Nearest neighbor pattern classification." IEEE Trans. Information Theory, 13(1), 21-27.
- Quinlan, J. R. (1986). "Induction of Decision Trees." Machine Learning, 1(1), 81-106.